



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

GROUP PROJECT

Fixed & Dynamic Memory Management – Next Fit & Worst Fit

**SECR2043 OPERATING SYSTEM
(SISTEM PENGOPERASIAN)**

SECTION 02

Lecturer Name:

Dr. Muhammad Zafran bin Muhammad Zaly Shah

Group Name:

Mushroom Soup

Group Members:

Name	Matric Number
CHAU YING JIA	A23CS0213
SABRINA HENG WEI QI	A23CS0265
GUI KAH SIN	A23CS0080
NG YU HIN	A23CS0148

Video:

TABLE OF CONTENTS

1.0 Title & Abstract.....	3
2.0 Introduction.....	3
3.0 Problem Statement.....	4
4.0 Related Works.....	5
5.0 Methodology.....	6
6.0 Design & Implementation.....	7
6.1 System Architecture.....	7
6.2 Allocation Logic.....	7
6.3 Implementation Details.....	8
6.4 System Flowchart.....	8
6.5 Simulation Execution.....	9
6.6 Code Implementation.....	10
7.0 Results & Discussion.....	12
7.1 Fixed Partitioning Results.....	13
7.2 Dynamic Partitioning Results.....	14
8.0 Future Work & Conclusion.....	18
9.0 Acknowledgement.....	19
10.0 References.....	19

1.0 Title & Abstract

Memory Management Visualization: Simulating Next Fit and Worst Fit Allocation Strategies

Memory management is a fundamental function of an operating system (OS), responsible for efficiently allocating memory to processes. Among various allocation strategies, Next Fit and Worst Fit are two commonly used dynamic memory allocation algorithms with differing performance in fragmentation and efficiency. Next Fit assigns memory to the next suitable free block starting from the last allocated position, while Worst Fit searches the entire memory space and allocates the largest available block to the requesting process. Each strategy has different effects on space utilization and fragmentation.

To improve the understanding of these strategies and their real-world implications, this project applies a parking lot management system as an analogy for memory allocation. In this simulation, cars of varying lengths represent processes and parking slots represent available memory blocks. The simulation models both fixed and dynamic partitioning environments to observe how each strategy behaves using both algorithms, we assess allocation success, internal and external fragmentation and overall slot utilization. This real-world analogy not only makes the technical concept more accessible but also provides valuable insight into how allocation strategies affect memory performance and efficiency in practice.

2.0 Introduction

Memory management is one of the most fundamental and vital aspects of an operating system. It involves the handling of memory allocation for different processes during their execution. The main goal is to ensure efficient utilization of system memory, avoid fragmentation and reduce the chance of process rejection due to lack of space. One of the challenges in memory management is how to place incoming processes into available memory blocks in a way that minimizes wasted space and maximises memory usage.

Contiguous memory allocation strategies are among the simplest and most straightforward approaches. In this context, Next Fit and Worst Fit are two well-known algorithms. The Next

Fit algorithm works by scanning the memory from the last allocated block and assigning the next available block that is large enough for the process. The Worst Fit strategy, in contrast, searches the entire memory and allocates the process to the largest available block, under the assumption that splitting larger blocks results in fewer unusable small fragments later on.

To visualize and relate this abstract concept to something more concrete, this project adopts a real-world analogy of parking lot management. In the analogy, a parking lot represents system memory and cars represent processes. Cars arrive with specific size requirements and the parking lot manager (representing the OS) must assign each to a suitable parking slot. A fixed partitioning model assumes that the parking slots are of fixed sizes, while a dynamic model assumes that parking slots can be split and adjusted depending on the vehicle's size. The logic used to assign vehicles to slots reflects how memory allocation strategies operate in a computer system.

Through this analogy, learners can better understand how memory is managed in real systems and how allocation strategies impact fragmentation and resource utilization. The project also aims to compare both algorithms under different simulated parking conditions to evaluate which strategy performs more effectively in terms of space usage and allocation success.

3.0 Problem Statement

Efficient memory allocation is essential in ensuring that the operating system can execute processes without unnecessary delays or resource wastage. However, when memory is allocated without proper strategy, it can lead to fragmentation, where memory is available but not in the right sizes or locations to meet the demands of incoming processes. This can prevent new processes from being loaded even when there is technically enough free space. Therefore, choosing an appropriate memory allocation strategy is vital to achieving high performance and low memory wastage in any multitasking system.

This project explores this problem using a real-world scenario of parking lot management. In this analogy, the parking lot represents system memory and cars of different sizes represent processes requiring memory blocks. As cars arrive at the parking lot, they must be assigned to available parking slots. If the parking slots are not chosen wisely, large areas of space can become unusable for incoming cars, even though they are technically free. This is directly comparable to external fragmentation in operating systems.

For example, if a small car is parked in a large slot early on, a bigger car arriving later may not find a space that fits, leading to allocation failure. Similarly, if the parking manager always skips around looking for the largest empty slot, it may delay the parking process and still leave behind scattered small gaps that no car can use. These two approaches mirror the Next Fit and Worst Fit memory allocation algorithms respectively.

The key problem this project investigates is: how do Next Fit and Worst Fit allocation strategies perform under different conditions when applied to real-world inspired environments such as parking lots? This includes assessing:

- How many processes (cars) can be successfully allocated (parked),
- How much space is wasted due to fragmentation, and
- Which method performs better in both fixed and dynamic slot configurations.

By translating the technical problem into a practical and observable environment, the project aims to offer a more intuitive and visual way of understanding how operating systems manage memory and why the choice of strategy matters.

4.0 Related Works

A variety of memory allocation strategies have been extensively studied as part of contiguous memory management in operating systems. We focus on Next-fit and Worst-Fit, two well-known dynamic memory allocation strategies. Next-Fit searches for memory allocation starting from the point of the last successful allocation while Worst-Fit selects the largest available memory block to accommodate a new process aiming to reduce fragmentation by leaving behind larger free holes. In this project, we only cover these two strategies out of four, the other two are the Best-Fit and First-Fit. These strategies are part of the dynamic partitioning model where memory blocks vary in size and can be assigned as needed, unlike fixed partitioning where block sizes are predetermined and inflexible.

Theoretically, Next-Fit reduces the search overhead but may lead to inefficient allocation while Worst-Fit selects the largest blocks to accommodate new processes that will inefficiently be used for incoming processes. These trade-offs have motivated various educational tools and simulations that help visualize memory behaviour. In this project, we

have employed real-world analogies which are the parking lots as a way to teach memory allocation more intuitively. The parking lot analogy in particular is widely appreciated because it mirrors memory fragmentation with unused parking wasted, total parking space used and allocation failures due to a variety of different sizes of vehicles. This analogy aligns with our project's aim to model processes as cars and memory blocks as parking slots, offering an engaging and visual understanding of abstract operating system concepts.

By building on these well-established strategies and educational models, our project contributes a hands-on simulation that not only reinforces theoretical knowledge but also supports experiential learning through real-world analogies.

5.0 Methodology

In approaching this project, our group adopted a real-world analogy to simulate the behaviour of memory allocation strategies under two distinct scenarios which are fixed and dynamic partitioning schemes. We chose the parking lot simulation, where each car represents a process and each parking slot represents a memory block. This analogy helped us visualise how processes of varying sizes are placed into available memory spaces and how fragmentation can occur depending on the allocation strategy used.

To ensure consistency and fair comparability, we tested both partitioning models separately using the same set of input processes, represented by cars of different sizes. In the fixed partitioning model, memory was divided into predefined, static block sizes, and allocations were attempted using both Next Fit and Worst Fit approaches. In contrast, the dynamic partitioning model treated memory as a single contiguous block where allocations were made based on exact process size, with the system dynamically adjusting block boundaries. When a process completed and its memory was released, holes were created and new processes were subsequently allocated into those holes based on their respective strategies.

Throughout the simulation, we qualitatively observed differences in memory allocation behaviour such as skipped blocks, leftover spaces, and the formation of fragmentation. By simulating both approaches in both memory models, we could gain valuable insight into how memory management decisions affect allocation outcomes, fragmentation patterns, and overall system efficiency.

6.0 Design & Implementation

This project was implemented using the Python programming language, selected for its simplicity and suitability for logic-based simulations. The core of our system is a simulation that models memory allocation under two different memory management schemes – Fixed Partitioning and Dynamic Partitioning. For each scheme, we independently tested two placement algorithms – Next Fit and Worst Fit. The simulator takes a series of incoming processes (cars), processes them based on the selected memory model and strategy, and provides detailed step-by-step allocation outputs along with the final memory state after processes terminate.

6.1 System Architecture

The simulator was designed with two independent modules to represent the Fixed Partitioning and Dynamic Partitioning memory models. The first module simulates a fixed partitioning model, where memory is represented as a list of static blocks corresponding to parking lots of predefined and immutable sizes. Each process (car) is entered one by one, and the system attempts to allocate it immediately using the selected strategy. After each allocation attempt, the simulator prints which spot was used, allowing users to see exactly where each car parks. At the end of the simulation, the system will then display the final parking lot state.

In contrast, the second module simulates a dynamic partitioning model, where memory is treated as a contiguous block. Each allocation splits the memory exactly according to the process size, and when a process exits, its memory is marked as a free hole. The simulator then attempts to reuse these holes based on the chosen strategy. The output includes the result of each process and a final parking lot state showing the free blocks. This modular design allows consistent testing and comparison of allocation outcomes across both memory models.

6.2 Allocation Logic

Each strategy was applied separately under each memory model to observe how they handle placement decisions and fragmentation.

- Next Fit:

This strategy begins its search from the location of the last successful allocation and then proceeds sequentially. If the end of the memory is reached, it wraps around to the beginning. It offers improved speed by avoiding a full scan but may skip earlier suitable blocks, potentially causing allocation failures even when enough memory exists.

- Worst Fit:
This strategy searches the entire memory space and selects the largest available block or hole for each allocation. The theoretical goal is to reduce fragmentation by leaving behind larger usable spaces that can still accommodate future processes. However, in practice, it can lead to inefficient use of large blocks, as smaller processes may consume them early, preventing larger processes from being allocated later.

6.3 Implementation Details

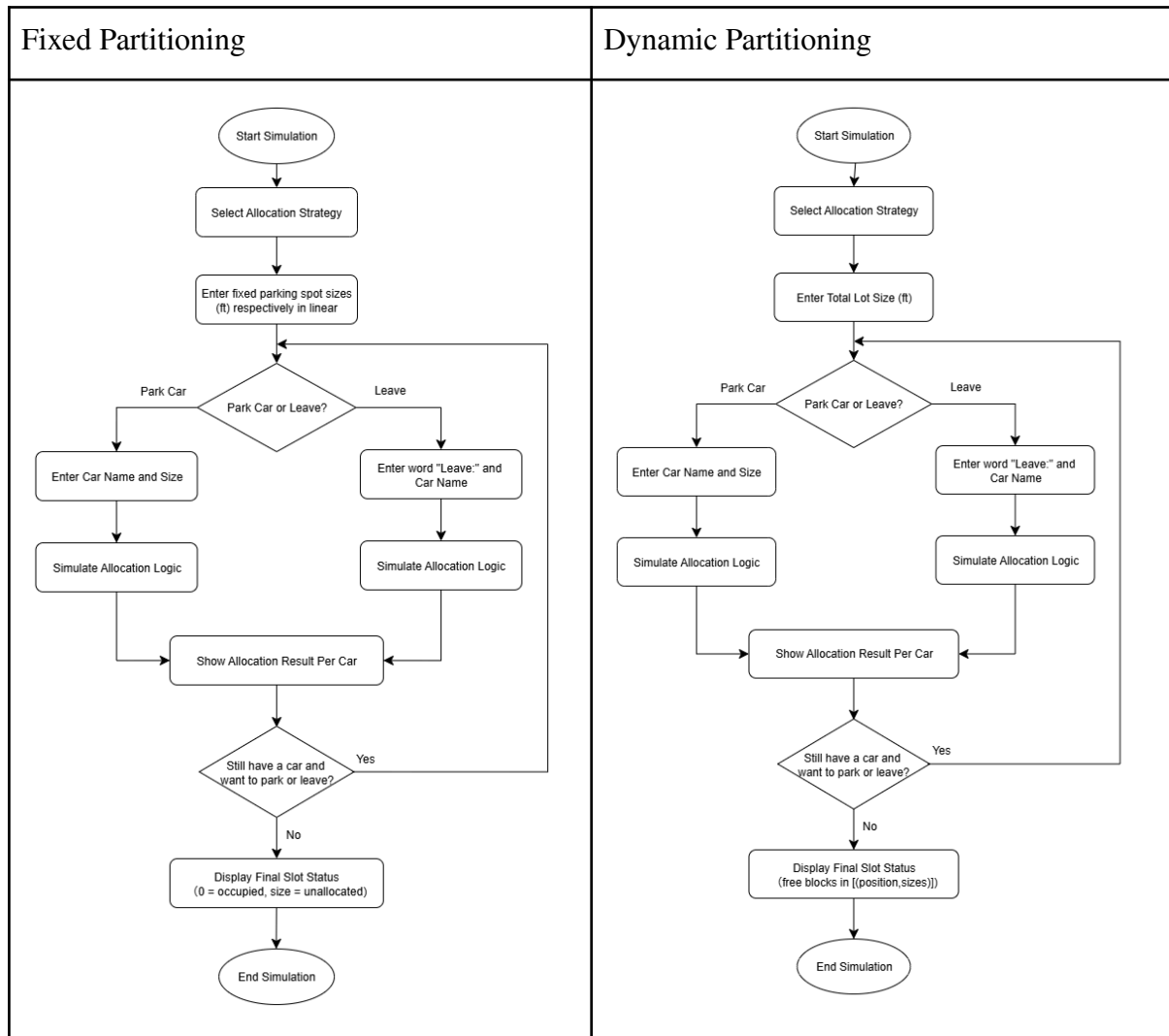
The simulator was implemented using basic Python code, supported by an Animaker animation to visually represent the allocation process.

- In the fixed partitioning module, memory is defined as a list of static block sizes representing fixed parking lots. The simulator processes each incoming car one by one, immediately attempting to allocate it to a suitable block according to the strategy. After each allocation, the system indicates whether the allocation was successful and specifies which parking slot was used, or it notes if the allocation failed. The output is displayed as a final allocation result, where occupied slots are marked as 0 and unallocated slots retain their original size.
- In the dynamic partitioning module, memory is modelled as a single contiguous block. For each allocation, the system calculates the memory range used by a process and outputs a statement indicating which positions have been occupied and which have been free. The final output displayed after processes terminate is the final parking lot state, which indicates free blocks with their sizes.

The blocks (representing the parking lots) are assigned specific size values; however, in the fixed model, there are distinct sizes for parking slots, whereas the dynamic model utilises a total size. The processes (cars) are defined with size values and processed in the order they are entered by the user. The system does not maintain a coded status flag for each block or segment. Instead, it communicates allocation outcomes through textual updates and the final memory state, helping users visualise how memory is utilised or fragmented under each model and strategy. The use of Animaker further enhances understanding by animating how cars enter, occupy space, and leave, making the concept of memory management more intuitive and easier to understand.

6.4 System Flowchart

The following flowchart represents the general logic of the memory allocation simulation applied to both the Fixed and Dynamic Partitioning modules. It outlines the process from initialisation to result output, guided by the selected allocation strategy.



6.5 Simulation Execution

The simulation was executed through a Python command-line interface, allowing user interaction based on the selected memory model and strategy.

- For Fixed Partitioning, the user enters:
 - A list of parking slot sizes to initialize the parking lot
 - Cars are entered one by one with their names and sizes
 - System immediately displays which parking slot the car was allocated to (or if it failed)

- The system display final parking lot state after all processes terminate
- For Dynamic Partitioning, the user enters:
 - Specifies the total parking lot size
 - Car commands one-by-one with name and size
 - The system display which parking space of parking lot is allocated or deallocated
 - Displays final parking lot used with free block when the user types “done”

This setup allowed consistent comparison of both memory models and strategies under a controlled set of inputs, highlighting allocation success and fragmentation behaviour.

6.6 Code Implementation

```
class Car:
    def __init__(self, name, size):
        self.name = name
        self.size = size

class DynamicPartitioning:
    def __init__(self, total_size):
        self.total_size = total_size
        self.free_blocks = [(0, total_size)]
        self.allocated_blocks = {}
        self.last_index = 0

    def allocate_next_fit(self, car):
        n = len(self.free_blocks)
        for i in range(n):
            index = (self.last_index + i) % n
            start, size = self.free_blocks[index]
            if size >= car.size:
                self.allocated_blocks[car.name] = (start, car.size)
                new_start = start + car.size
                new_size = size - car.size
                if new_size > 0:
                    self.free_blocks[index] = (new_start, new_size)
                else:
                    self.free_blocks.pop(index)
            self.last_index = index
            print(f"Car {car.name} ({car.size}ft) parked at position {start}")
            return
        print(f"Car {car.name} ({car.size}ft) could not be parked.")

    def allocate_worst_fit(self, car):
        max_index = -1
        max_size = -1
        for i, (start, size) in enumerate(self.free_blocks):
            if size >= car.size and size > max_size:
                max_size = size
                max_index = i
        if max_index != -1:
            start, size = self.free_blocks[max_index]
            self.allocated_blocks[car.name] = (start, car.size)
```

```

        new_start = start + car.size
        new_size = size - car.size
        if new_size > 0:
            self.free_blocks[max_index] = (new_start, new_size)
        else:
            self.free_blocks.pop(max_index)
        print(f"Car {car.name} ({car.size}ft) parked at position {start}")
    else:
        print(f"Car {car.name} ({car.size}ft) could not be parked.")

    def deallocate(self, car_name):
        if car_name in self.allocated_blocks:
            start, size = self.allocated_blocks.pop(car_name)
            self.free_blocks.append((start, size))
            self.free_blocks.sort()
            self.merge_free_blocks()
            print(f"Car {car_name} left. Space {start}-{start + size}ft is now free.")
        else:
            print(f"Car {car_name} is not parked.")

    def merge_free_blocks(self):
        merged = []
        for block in sorted(self.free_blocks):
            if merged and merged[-1][0] + merged[-1][1] == block[0]:
                last_start, last_size = merged.pop()
                merged.append((last_start, last_size + block[1]))
            else:
                merged.append(block)
        self.free_blocks = merged

    def print_free_blocks(self):
        print("\nFinal Parking Lot State (Free blocks):", self.free_blocks)

class FixedPartitioning:
    def __init__(self, spots):
        self.original_sizes = spots[:]
        self.spots = spots[:]
        self.allocated = [None] * len(spots)
        self.last_index = 0

```

```

    def next_fit(self, cars):
        for car in cars:
            allocated = False
            scanned = 0
            n = len(self.spots)
            while scanned < n:
                i = (self.last_index + scanned) % n
                if self.allocated[i] is None and self.spots[i] >= car.size:
                    self.allocated[i] = car.name
                    self.spots[i] -= car.size
                    self.last_index = (i + 1) % n #Pointer moves to NEXT slot
                    print(f"Car {car.name} ({car.size}ft) parked in spot {i + 1}")
                    allocated = True
                    break
                scanned += 1
            if not allocated:
                print(f"Car {car.name} ({car.size}ft) could not be parked.")
        print("\nFinal Parking Lot State:", self.spots)

    def worst_fit(self, cars):
        for car in cars:
            max_index = -1
            max_size = -1
            for i in range(len(self.spots)):
                if self.allocated[i] is None and self.spots[i] >= car.size and self.spots[i] > max_size:
                    max_size = self.spots[i]
                    max_index = i
            if max_index != -1:
                self.allocated[max_index] = car.name
                self.spots[max_index] -= car.size
                print(f"Car {car.name} ({car.size}ft) parked in spot {max_index + 1}")
            else:
                print(f"Car {car.name} ({car.size}ft) could not be parked.")
        print("\nFinal Parking Lot State:", self.spots)

    def deallocate(self, car_name):
        for i in range(len(self.allocated)):
            if self.allocated[i] == car_name:
                print(f"Car {car_name} left. Spot {i + 1} is now free.")
                self.allocated[i] = None

```

```

        self.spots[i] = self.original_sizes[i]
        return
    print(f"Car {car_name} is not found.")

def main():
    print("Fixed & Dynamic Memory Management - Next Fit & Worst Fit")
    print("=== Parking Lot Simulation ===")
    print("\nChoose allocation strategy:")
    print("1. Fixed - Next Fit")
    print("2. Fixed - Worst Fit")
    print("3. Dynamic - Next Fit")
    print("4. Dynamic - Worst Fit")
    choice = input("Enter choice (1-4): ").strip()

    if choice in ['1', '2']:
        lot_input = input("Enter fixed parking spot sizes (e.g., 20,15,10): ")
        spots = list(map(int, lot_input.strip().split(',')))
        fp = FixedPartitioning(spots)
        while True:
            cmd = input("Enter command (e.g., A:10,B:15 or leave:A or done): ").strip()
            if cmd == 'done':
                break
            elif cmd.startswith('leave:'):
                _, name = cmd.split(':')
                fp.deallocate(name.strip())
            else:
                try:
                    cars = []
                    for part in cmd.strip().split(','):
                        name, size = part.split(':')
                        cars.append(Car(name.strip(), int(size.strip())))
                    if choice == '1':
                        fp.next_fit(cars)
                    else:
                        fp.worst_fit(cars)
                except:
                    print("Invalid command.")
    elif choice in ['3', '4']:
        total_size = int(input("Enter total dynamic parking lot size (ft): "))
        dp = DynamicPartitioning(total_size)

```

```

        while True:
            cmd = input("Enter command (e.g., A:10, leave:A, done): ").strip()
            if cmd == 'done':
                break
            elif cmd.startswith('leave:'):
                _, name = cmd.split(':')
                dp.deallocate(name.strip())
            else:
                try:
                    name, size = cmd.split(':')
                    car = Car(name.strip(), int(size.strip()))
                    if choice == '3':
                        dp.allocate_next_fit(car)
                    else:
                        dp.allocate_worst_fit(car)
                except:
                    print("Invalid command.")
            dp.print_free_blocks()
        else:
            print("Invalid choice.")

if __name__ == "__main__":
    main()

```

7.0 Results & Discussion

This section will present the outcome of the parking lot simulation based on the two memory allocation strategies, Next Fit and Worst Fit across both Fixed Partitioning and Dynamic Partitioning environments. Each configuration was tested using predefined parking lot sizes and a set of incoming cars with different space requirements. The analysis focuses on three main criteria:

- Allocation success rate: how many cars parked successfully
- Space utilization: how much total space used
- Fragmentation: how much memory wasted

7.1 Fixed Partitioning Results

In the fixed allocation model, the parking lot consisted of predefined slot sizes: [20, 10, 15, 40]. Cars with sizes [10, 20, 30, 40] were processed using both allocation strategies.

Next Fit

```
rick@secr2043:~$ nano parking_demo3.py
rick@secr2043:~$ chmod +x run_demo3.sh
rick@secr2043:~$ ./run_demo3.sh
Fixed & Dynamic Memory Management - Next Fit & Worst Fit
=== Parking Lot Simulation ===

Choose allocation strategy:
1. Fixed - Next Fit
2. Fixed - Worst Fit
3. Dynamic - Next Fit
4. Dynamic - Worst Fit
Enter choice (1-4): 1
Enter fixed parking spot sizes (e.g., 20,15,10): 15,10,20
Enter command (e.g., A:10,B:15 or leave:A or done): A:20
Car A (20ft) parked in spot 3

Final Parking Lot State: [15, 10, 0]
Enter command (e.g., A:10,B:15 or leave:A or done): B:5
Car B (5ft) parked in spot 1

Final Parking Lot State: [10, 10, 0]
Enter command (e.g., A:10,B:15 or leave:A or done): leave:B
Car B left. Spot 1 is now free.
Enter command (e.g., A:10,B:15 or leave:A or done): C:10
Car C (10ft) parked in spot 2

Final Parking Lot State: [15, 0, 0]
Enter command (e.g., A:10,B:15 or leave:A or done): done
```

Next Fit searches from the last allocated slot and continues forward. In fixed partitioning, memory blocks are predefined, so the system checks them in order after the last allocation.

In the simulation:

- Car A (10ft) goes to Slot 1 (20ft → 10ft left)
- Car B (20ft) goes to Slot 4 (40ft → 20ft left)
- Car C (30ft) is not parked because Next Fit resumes from Slot 4, wraps around to Slot 1, but does not find a slot with 30ft or more
- Car D (40ft) also cannot be parked

According to this output, we can conclude that cars were placed in order but some suitable space was skipped due to the linear search. The final slot state shows leftover memory, but allocation failed due to search behavior, not lack of space.

Worst Fit

```
rick@secr2043:~$ ./run_demo3.sh
Fixed & Dynamic Memory Management - Next Fit & Worst Fit
=== Parking Lot Simulation ===

Choose allocation strategy:
1. Fixed - Next Fit
2. Fixed - Worst Fit
3. Dynamic - Next Fit
4. Dynamic - Worst Fit
Enter choice (1-4): 2
Enter fixed parking spot sizes (e.g., 20,15,10): 15,10,20
Enter command (e.g., A:10,B:15 or leave:A or done): A:20
Car A (20ft) parked in spot 3

Final Parking Lot State: [15, 10, 0]
Enter command (e.g., A:10,B:15 or leave:A or done): B:5
Car B (5ft) parked in spot 1

Final Parking Lot State: [10, 10, 0]
Enter command (e.g., A:10,B:15 or leave:A or done): leave:B
Car B left. Spot 1 is now free.
Enter command (e.g., A:10,B:15 or leave:A or done): C:10
Car C (10ft) parked in spot 1

Final Parking Lot State: [5, 10, 0]
Enter command (e.g., A:10,B:15 or leave:A or done): done
```

Worst Fit scans all slots and chooses the largest one for each car. The idea is to leave behind bigger pieces of memory for future processes.

In the simulation:

- Car A (10ft) is placed in Slot 4 (40ft → 30ft left)
- Car B (20ft) is placed in Slot 1 (20ft → 0ft left)
- Car C (30ft) is not parked, even though Slot 4 has exactly 30ft
- Car D (40ft) is too large for any slot

According to this output, we can conclude that the large blocks were used early for small cars. When large cars arrived, there was no suitable space for it. Therefore, the output reflects inefficient long-term planning.

7.2 Dynamic Partitioning Results

For dynamic partitioning, memory is treated as one continuous block of 105ft. Memory is allocated in exact sizes based on the car's requirements and fragmentation occurs only when blocks are freed and not reused effectively.

Next Fit

```
rick@secur2043:~$ ./run_demo3.sh
Fixed & Dynamic Memory Management - Next Fit & Worst Fit
=== Parking Lot Simulation ===

Choose allocation strategy:
1. Fixed - Next Fit
2. Fixed - Worst Fit
3. Dynamic - Next Fit
4. Dynamic - Worst Fit
Enter choice (1-4): 3
Enter total dynamic parking lot size (ft): 35
Enter command (e.g., A:10, leave:A, done): A:5
Car A (5ft) parked at position 0
Enter command (e.g., A:10, leave:A, done): B:10
Car B (10ft) parked at position 5
Enter command (e.g., A:10, leave:A, done): C:20
Car C (20ft) parked at position 15
Enter command (e.g., A:10, leave:A, done): leave:A
Car A left. Space 0-5ft is now free.
Enter command (e.g., A:10, leave:A, done): leave:C
Car C left. Space 15-35ft is now free.
Enter command (e.g., A:10, leave:A, done): A:5
Car A (5ft) parked at position 0
Enter command (e.g., A:10, leave:A, done): done

Final Parking Lot State (Free blocks): [(15, 20)]
```

Dynamic Next Fit starts searching for holes from the last allocated position. Memory is a continuous block and each allocation splits it based on exact process size.

In the simulation:

- Cars A to D fill memory from 0 to 95ft
- When A and D leave, holes are created
- Next Fit starts from last position and allocates:
 - E (5ft) → 0–5 (from A's hole)
 - F (30ft) → 45–75 (from D's hole)
 - G (10ft) → 25–35 (from C's hole)

According to the output, it shows that Next Fit reuses space in order but not necessarily the most efficient place. Holes like (5-10), (35-45) and (75-105) are left behind.

Worst Fit

```
rick@secr2043:~$ ./run_demo3.sh
Fixed & Dynamic Memory Management - Next Fit & Worst Fit
=== Parking Lot Simulation ===

Choose allocation strategy:
1. Fixed - Next Fit
2. Fixed - Worst Fit
3. Dynamic - Next Fit
4. Dynamic - Worst Fit
Enter choice (1-4): 4
Enter total dynamic parking lot size (ft): 35
Enter command (e.g., A:10, leave:A, done): A:5
Car A (5ft) parked at position 0
Enter command (e.g., A:10, leave:A, done): B:10
Car B (10ft) parked at position 5
Enter command (e.g., A:10, leave:A, done): C:20
Car C (20ft) parked at position 15
Enter command (e.g., A:10, leave:A, done): leave:A
Car A left. Space 0-5ft is now free.
Enter command (e.g., A:10, leave:A, done): leave:C
Car C left. Space 15-35ft is now free.
Enter command (e.g., A:10, leave:A, done): A:5
Car A (5ft) parked at position 15
Enter command (e.g., A:10, leave:A, done): done

Final Parking Lot State (Free blocks): [(0, 5), (20, 15)]
```

Dynamic Worst Fit chooses the largest available hole for each car. After some cars leave, the system scans all holes and places new processes in the biggest space.

In the simulation:

- Cars A to D fill memory from 0 to 95ft
- A and D leave, holes created
- Worst Fit allocates:
 - E (5ft) → 45–50
 - F (30ft) → 50–80
 - G (10ft) → 80–90

According to the output, it shows holes are better grouped with fewer small gaps. Fragmentation is less scattered and the allocation is smarter in terms of long-term reuse.

7.3 Differences of Next Fit and Worst Fit

The Next Fit and Worst Fit allocation strategies differ significantly in how they search for available memory and manage fragmentation. In Next Fit, the allocation process begins searching from the position of the last successful allocation and proceeds linearly through the memory blocks or holes. Once it reaches the end of memory, it wraps around to the beginning if necessary. This strategy emphasizes speed and simplicity, making it suitable for systems that prioritize quick memory assignment. However, due to its sequential search nature, Next Fit may miss earlier suitable memory blocks, which can lead to allocation failures even when enough memory is technically available.

In contrast, Worst Fit always performs a full scan of the available memory and selects the largest available block for allocation. The goal is to leave behind larger leftover spaces under the assumption that this approach will reduce fragmentation in the long run. This method can be more memory-efficient in some cases, especially in dynamic partitioning, as it tends to produce fewer, larger holes that are more reusable for future processes. However, in fixed partitioning, Worst Fit performs poorly because small processes may consume the largest slots early.

Overall, Next Fit favors allocation speed and locality, while Worst Fit focuses on space preservation and fragmentation control. The choice between them depends on the system's goals—whether to prioritize faster allocations (Next Fit) or better long-term memory efficiency (Worst Fit).

8.0 Future Work & Conclusion

This project successfully simulated and compared two widely known memory allocation strategies – Next Fit and Worst Fit across both fixed and dynamic memory partitioning models. By applying a real-world parking lot analogy, the project made abstract concepts more accessible and easier to understand. The simulation demonstrated how different strategies impact allocation, space utilization, and fragmentation.

Throughout the study, we observed that:

- Next Fit performs faster due to its linear search method but it may miss better-fit spaces, leading to allocation failures even when space is available
- Worst Fit often leaves behind smaller, unusable holes in fixed models but proves more effective in dynamic scenarios due to its space-maximizing approach.
- Dynamic partitioning allows more flexible and efficient memory allocation compared to fixed partitioning, which is more prone to internal fragmentation.

Despite the usefulness of the current implementation, limitations do exist. The simulation is purely text-based and lacks real-time visualization or performance metrics like fragmentation percentage.

Therefore, in the future, the simulation could be enhanced by

- Integrating a graphical user interface (GUI) to visualize memory allocation in real time
- Adding automated performance analysis, including fragmentation reports and memory usage efficiency.
- Supporting more allocation strategies like First Fit and Best Fit for broader comparison.

In conclusion, this project has provided valuable insights into how different memory management strategies function and affect overall system performance. It reinforces the importance of selecting appropriate placement algorithms based on the system's needs and highlights how visual analogies can significantly improve conceptual understanding in operating system studies.

9.0 Acknowledgement

We would like to express our heartfelt appreciation to our respected lecturer, Dr. Muhammad Zafran Bin Muhammad Zaly Shah, for his invaluable guidance, insightful advice, and steadfast support throughout this project. Dr. Zafran's deep knowledge, clear explanations, and passionate teaching have been crucial in expanding our understanding of this complex subject. His thoughtful feedback and constant encouragement motivated us to strive for excellence and greatly contributed to the successful completion of our work. We are especially grateful for his dedication to our academic and professional development, which not only strengthened our technical skills but also deepened our appreciation of the practical applications of operating system concepts in real-world scenarios. It has been a privilege to learn from him, and we truly appreciate the time, patience, and insight he has shared with us. This project would not have been possible without his continued support. Thank you, Dr. Zafran, for being an inspiring mentor and for your remarkable contribution to our learning journey.

10.0 References

- Silberschatz, A., Galvin, P. B., & Gagne, G. (2018). *Operating system concepts* (10th ed.). Wiley.
- GeeksforGeeks. (n.d.). *Memory allocation algorithms*. Retrieved [30.6.2025], from <https://www.geeksforgeeks.org/memory-management-in-operating-system/>
- TutorialsPoint. (n.d.). *Operating system - memory management*. Retrieved [30.6.2025], from https://www.tutorialspoint.com/operating_system/os_memory_management.htm
- Python Software Foundation. (n.d.). *Python documentation*. Retrieved [30.6.2025], from <https://docs.python.org/3/>